Java Script: Object

Playground: https://www.programiz.com/javascript/online-compiler/

Basic Understanding

- JavaScript is an object-based language: Almost everything is an object
- Variables are container for storing data value
- Objects are variables too. But object can contain many values.
  - object basically a container that hold key-value pairs (also called properties).

example:

```
const person = {
    name: "Robert Cray",
    car: "Tesla Model Y",
    age: 72,
    isAlive: true
}
```

Object can hold methods

example:

```
const car = {
    name: "Range Rover",
    model: "2025",
    color: "yellow",
    drive: function(){
        console.log("driving...")
    },
    stop: function(){
        console.log("stoping...")
    }
}
```

Accessing and adding properties

```
const person = {
    firstName: "Bryan",
    lastName: "Wilson",
    age: 50
}

// Accessing and reasign property using dot notation
console.log(person.age)
person.age = 25
console.log(person.age)

// Accessing and reasign property using bracket notation
console.log(person["firstName"])
person["firstName"] = "Jonny"
console.log(person["firstName"])

// Adding property
console.log(person)
person.salary = 21000
person["home"] = "Alaska"
console.log(person);

// Accessing nonexist property
console.log(person.school)
```

Accessing object's member with `this` keyword

Create a coach object that contain firsName, lastName, and age properties and sayFullName() method

```
const coach = {
    firstName: "Bryan",
    lastName: "Wilson",
    age: 34,
    sayFullName: function() {
        console.log(this.firstName + " " + this.lastName)
```

```
    }
}

coach.sayFullName()

// JavaScript is dynamic
coach["sayFullName"]()
const functionNameToCall = "sayFullName"
coach[functionNameToCall]()
```

Using `for...in` loop

This loop iterates over all enumerable properties of an object

```
const coach = {
    firstName: "Bryan",
    lastName: "Wilson",
    age: 34,
    sayFullName: function() {
        console.log(this.firstName + " " + this.lastName)
    }
}

for (key in coach) {
    console.log(key)
}

console.log("----------------------")
for (key in coach) {
    console.log(typeof key)
}

console.log("----------------------")
for (key in coach) {
    console.log(coach[key])
}

console.log("----------------------")
```

```
for (key in coach) {
    console.log(typeof coach[key])
}
```

Object Destructuring

```
const coach = {
    firstName: "Bryan",
    lastName: "Wilson",
    age: 34,
    location: {
        city: "BKK",
        country: "Thailand"
    }
}

function message(personObj) {
    console.log("Hello, " + personObj.firstName + " " + personObj.lastName);
}

function shorter(personObj) {
    const { firstName, lastName } = personObj;
    console.log("Hello, " + firstName + " " + lastName);
}

function moreShorter({ firstName, lastName }) {
    console.log("Hello, " + firstName + " " + lastName);
}

function smartAndShort({ firstName, lastName, location: { country }}) {
    console.log(`Hello ${firstName} ${lastName}. Are you from ${country}`)
}

message(coach);
shorter(coach);
moreShorter(coach);
smartAndShort(coach);
```

Object and Spread Operator

- The JavaScript spread operator (`...`) is used to expand or spread out elements of an iterable, such as an `array`, `string`, or `object`.
- This makes it incredibly useful for tasks like combining objects, passing elements to functions as separate arguments, or even copying objects.

Example:

```
let obj1 = { x : 1, y : 2 };
let obj2 = { z : 3 };

// add obj1 and obj2 without spread operator
let obj3 = {obj1, obj2};

// use the spread operator to add
// members of obj1 and obj2 to obj3
let obj4 = {...obj1, ...obj2};

console.log("obj3 =", obj3);
console.log("obj4 =", obj4);
```

JavaScript Rest Parameter

When the spread operator is used as a parameter, it is known as the rest parameter.

You can accept multiple arguments in a function call using the rest parameter. For example,

```
let printArray = function(...args) {
    console.log(args);
}

// pass a single argument
printArray(3);
```

```
// pass multiple arguments
printArray(4, 5, 6);
```

> *When three arguments are passed, the rest parameter (`args`) takes all three parameters as array elements.*

Spread Operator as Part of Function Argument

Example:

```
// function that takes three arguments
function sum(num1, num2 , num3) {
    console.log(num1 + num2 + num3);
}

let num1 = [1, 3, 4, 5];

// pass the first three array elements
sum(...num1);

// Output: 8
```

> *If you pass multiple arguments using the spread operator, the function takes the required number of arguments and ignores the rest.*

Object.key() and Object.value()

```
const coach = {
    firstName: "Bryan",
    lastName: "Wilson",
    age: 34
}
```

```
console.log(Object.keys(coach));
console.log(Object.values(coach));
```

JavaScript Object and JSON

```
const coach = {
    firstName: "Bryan",
    lastName: "Wilson",
    age: 34,
    sayFullName: function() {
        console.log(this.firstName + " " + this.lastName)
    }
}

console.log(coach)
console.log(JSON.stringify(coach))
console.log(JSON.parse(JSON.stringify(coach)))
```

JavaScript Objects VS JSON

JSON	JavaScript Object
The key in key/value pair should be in double quotes.	The key in key/value pair can be without double quotes.
JSON cannot contain functions.	JavaScript objects can contain functions.
JSON can be created and used by other programming languages.	JavaScript objects can only be used in JavaScript.

Exercise

```
const person = {
    firstName: "Julia",
    lastName: "Wilson",
    age: 34
}
```

```
// Create delKey() function that take two inputs,
// an object and a string of key name
// and return new  object without the specified key

const protectedPerson = delKey(person, "age");
console.log(protectedPerson);
//output: { firstName: 'Julia', lastName: 'Wilson' }
```

Solution

```
function delKey(obj, propertyName) {
    const { age, ...remain } = obj;
    return remain
}
```